VERÍDIA - O Enigma da Urna

Relatório técnico acadêmico - versão para revisão do estudante

IPT - Mestrado Profissional em Computação Aplicada

Disciplina: Estruturas de Dados e Análise de Algoritmos

Professora: Lucy Mari Tabuti

Data da consolidação: 06/11/2026

Este documento segue a organização sugerida na seção 16.1 do enunciado oficial. Foi preparado com apoio de IA a partir do código, dos testes e das medições desta rodada. A identificação acadêmica do autor, as considerações pessoais e a curadoria humana ainda precisam de revisão pelo estudante. Não representa aprovação ou validação humana já realizada.

Fonte normativa: EP Urna Lógica.pdf, 20 páginas, consultado em modo de leitura na pasta externa da disciplina. O original não foi copiado para o repositório. Sua impressão digital SHA-256 está em REQUISITOS_EP.md.

Artefatos de apoio: GUIA_TECNICO.md, REQUISITOS_EP.md, REGISTRO_IA.md e METRICAS_RODADA4.json. O último contém ambiente, cinco tempos por combinação, contadores, estados da AVL e um registro de árvore de busca.

1. Resumo do jogo e objetivo

VERÍDIA é um jogo educacional de investigação de uma eleição inteiramente fictícia. Uma falha de sincronização deixou registros fora de ordem e fragmentou a trilha de auditoria. O jogador organiza dados, movimenta evidências, percorre conexões condicionadas por pistas e verifica uma configuração final consistente. A vitória depende de regras verificáveis, sem candidatos, partidos ou resultados de eleições reais.

O objetivo computacional é integrar implementações próprias de fila, pilha, lista duplamente ligada, grafo, árvore AVL, Bubble Sort, Merge Sort, Quick Sort, BFS e DFS. A DFS também tem uma versão recursiva com rastro. O objetivo pedagógico é tornar observáveis as operações dessas estruturas e relacionar o comportamento dos algoritmos à corretude, à complexidade e ao desempenho medido.

A aplicação tem cinco fases integradas, uma sessão por navegador, selos de progressão, reinício de fase, histórico e desfazer. O modo Resolver executa buscas reais em instâncias independentes e permite acompanhar a ordem de exploração. A interface NiceGUI apresenta o mapa e a AVL usando o estado produzido pelo motor Python.

Na validação técnica, a suíte original de 183 testes passou. Um defeito concreto no grafo foi reproduzido, corrigido e protegido por um teste novo. A suíte passou com 185 testes, incluindo a validação do novo pacote de entrega. As funções de ordenação produziram a mesma ordem de chaves em três tamanhos e três cenários; caminhos encontrados pelas buscas foram reexecutados pelas regras. O cenário com energia 14 não admitiu solução nos três algoritmos.

O escopo desta entrega é um protótipo acadêmico local. O relatório da partida, exportado em JSON e HTML pela Fase 5, documenta uma execução do jogo. Este relatório acadêmico descreve sua construção e análise técnica.

2. História, fases, regras e condições de vitória

O jogador atua como Analista de Integridade de Dados da cidade fictícia de Verídia. Deve recuperar cinco selos de auditoria. As opções de votação são códigos inventados, como OPC-A, BRANCO e NULO. A narrativa serve de contexto para organização de dados e exploração de estados; não sustenta afirmações sobre eleições reais.

Fase	Regra e condição verificável de conclusão
1 - O Arquivo Embaralhado	Escolher algoritmo e critério; o lote só é aceito se estiver em ordem composta por zona, seção e horário, sem perda, duplicação ou alteração de registros.
2 - A Central de Triagem	Movimentar E1 a E6, respeitando FIFO e LIFO, para obter custódia E2, E4, E1, E6; fila e pilha ficam vazias e todas as evidências são contabilizadas uma vez.
3 - O Mapa das Seções	Percorrer arestas válidas, reunir pistas e chegar ao Tribunal com as evidências exigidas, sem exceder a energia disponível.
4 - O Cofre dos Estados	Classificar oito candidatos como novos ou repetidos; a AVL deve ficar balanceada e pelo menos uma rotação deve ser observada.
5 - A Auditoria Final	Reconferir o estado atual das fases, revalidar a trilha, executar o solver e ordenar evidências com algoritmo da Fase 1.

A sessão concede um selo ao verificar a conclusão da fase. A interface encaminha ações por ContextoTela.agir e não concede selos. O desbloqueio é sequencial. Reiniciar uma fase preserva selos anteriores, mas a auditoria final examina seu estado atual; por exemplo, reiniciar a custódia depois de obter o selo produz uma inconsistência até refazer a fase.

Ações recusadas fornecem feedback e preservam os dados operacionais. Na Fase 4, uma resposta errada aumenta explicitamente o contador de erros, sem inserir um estado incorreto na AVL. Na Fase 2, o desfazer restaura o retrato anterior das estruturas; na Fase 3, restaura o estado anterior ao movimento. As pistas do mapa são coletadas automaticamente na chegada.

O solver sugere uma sequência descoberta pela busca. Ele não aplica uma sequência fixa escondida no código. Na triagem, os sucessores são obtidos pelos mesmos métodos utilizados nas ações manuais. No mapa, ambos utilizam as funções de veridia/jogo/regras.py.

3. Modelagem dos dados, do grafo e dos estados

RegistroVotacao é um registro imutável com id, zona, seção, código da opção fictícia, horário e status. Os registros são armazenados em listas Python com acesso indexado. A validação verifica campos e unicidade de id. A geração utiliza um random.Random(seed) local; a seed padrão é 2026. O embaralhamento da Fase 1 também é reproduzível.

Evidencia possui id, tipo, descrição e estado. Mapa reúne vértices, grafo, evidências, início, destino, pistas necessárias e energia inicial. O mapa padrão possui nove vértices e doze conexões bidirecionais, acima do mínimo de oito e dez exigido. A lista de adjacência utiliza uma lista ligada por vértice, adequada ao mapa esparso e ao percurso de vizinhos.

As bifurcações e os ciclos permitem rotas alternativas. Cada aresta tem origem, destino, custo e, opcionalmente, porta e pistas exigidas. Quatro portas condicionam as conexões. A entrada no Arquivo pela Escola ou pela Ponte exige lacre_rompido. A passagem Ponte-Tribunal exige log_urna e registro_horario. Cartório-Tribunal exige também ata_secao.

O estado mínimo implementado é:

```
EstadoJogo = (posicao, pistas, portas_liberadas, energia)
chave = (posicao, tuple(sorted(pistas)),
         tuple(sorted(portas_liberadas)), energia)
```

Conjuntos de pistas e portas são normalizados para produzir a mesma chave independentemente da ordem de aquisição. Dois estados na mesma posição com pistas diferentes têm chaves distintas. A energia participa da chave, pois pode determinar se um mesmo caminho ainda é viável. Essa escolha amplia o número de estados, mas evita descartar configurações relevantes.

O grafo físico dos locais é explícito. O grafo de estados é implícito: gerar_sucessores produz apenas movimentos permitidos no estado atual. A vitória requer posição no destino e todas as pistas necessárias. Cada movimento reduz a energia por um custo positivo. Para um mapa e energia inicial fixos, o conjunto de estados alcançáveis é finito.

4. Arquitetura do software e divisão dos módulos

O motor usa a biblioteca padrão de Python e estruturas próprias. A interface depende de NiceGUI. As regras não dependem da representação visual. Não há banco de dados, autenticação, API paga ou serviço externo necessário para executar o jogo localmente.

Módulo	Responsabilidade
veridia/modelos/	Registros, evidências, estados e contrato do mapa.
veridia/dados/	Geração reproduzível e definição do mapa fictício.
veridia/estruturas/	Fila, pilha, lista dupla, AVL e grafo próprios.
veridia/algoritmos/	Três ordenações e métricas.
veridia/jogo/	Regras de movimento e sessão com progressão.
veridia/busca/	BFS, DFS iterativa/recursiva e reconstrução de caminho.
veridia/fases/	Controladores das cinco fases.
veridia/interface/	Rotas, componentes, SVG e exportação do relatório da partida.
veridia/demos/	Demonstrações e experimento de ordenação.
tests/	Testes de unidades, integração e usuário simulado.

O fluxo principal é:

```
Tela NiceGUI
    -> ContextoTela.agir(fase, acao, argumentos)
    -> SessaoJogo.executar
    -> controlador da fase
    -> regras / estruturas / ordenacao / solver
    -> ResultadoAcao + consultar()
    -> tela atualizada
```

SessaoJogo controla a fase corrente, os selos e o histórico em lista ligada. GerenciadorSessoes associa uma partida ao identificador do navegador. Reiniciar o servidor perde as partidas em memória; essa é uma limitação conhecida do protótipo local.

A página raiz organiza as rotas de Início, Jogar, cinco fases, Resolver, Instruções, Resultados e Sair. O evento veridia_local do mapa é registrado uma vez na raiz e encaminhado ao controlador da tela. O desenho SVG é derivado do mapa e da AVL reais, sem versões paralelas das regras.

5. Implementação das estruturas de dados

A fila encadeada mantém início e fim: enfileirar, desenfileirar e consultar frente têm custo O(1). A pilha encadeada mantém o topo: empilhar, desempilhar e consultar topo também custam O(1). Operações impossíveis em estruturas vazias geram exceção controlada, tratada pelo controlador quando aplicável à jogada.

A lista é duplamente ligada. Inserções e remoções nas extremidades são O(1); busca, acesso e alterações por posição custam O(n) no pior caso. O ponteiro anterior permite remover do fim sem percorrer a lista. A lista representa custódia, arquivo, histórico e adjacências do grafo.

A AVL armazena altura em cada nó, busca por chave comparável e reequilibra com rotações simples e duplas. Busca e inserção custam O(log n), contando comparações de chaves como O(1). Para chaves compostas maiores, o custo da comparação precisa ser considerado. No solver, o valor associado registra pai, ação e estado, permitindo controle de visitados e reconstrução.

5.1. Evidência de rotação, equivalência e busca

Na Fase 4, A e B são estados da Praça com energia 18; B possui a pista de autenticação. C equivale a A e não é reinserido. Ao inserir D, cuja posição é Torre, a ordem das chaves é A < B < D. O registro real do controlador mostra a árvore antes da inserção e depois do balanceamento:

```
Antes de inserir D       Depois da insercao e rotacao esquerda
    A                              B
     \                            / \
      B                          A   D

Transitorio reconstruido pelas chaves (antes da rotacao):
    A  h=3, fator=-2
     \
      B
       \
        D
```

O estado transitório é uma reconstrução didática da inserção, não uma captura adicional do motor. Depois da rotação, B tem altura 2 e fator 0; A e D têm altura 1 e fator 0. As árvores reais antes/depois estão em METRICAS_RODADA4.json. A inserção posterior de G demonstra rotação à direita. Os testes unitários também cobrem rotações duplas.

Após a classificação de A a H, buscar a chave de A retorna o rótulo A. Buscar o mesmo estado com energia 3 retorna encontrado=False. C e F são repetidos de A e D, respectivamente. A sequência final contém seis estados distintos. Os contadores de rotações duplas também incrementam as rotações simples componentes; eles não devem ser somados como eventos independentes.

6. Implementação e comparação dos algoritmos

Bubble Sort interrompe a execução quando uma passada não efetua trocas. Merge Sort divide o intervalo recursivamente e escolhe o elemento da esquerda em caso de empate. Ambos são estáveis. Quick Sort usa partição de Hoare e pivô por mediana de três; recorre somente no lado menor e percorre o maior por laço, limitando a pilha recursiva. Quick Sort não é estável.

Algoritmo	Melhor tempo	Tempo médio	Pior tempo	Memória do núcleo
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n) de pilha nesta implementação

A API faz cópia da entrada; portanto todas as funções têm também uma alocação O(n), mesmo quando o núcleo é in-place. Uma comparação conta uma comparação entre duas chaves; uma movimentação conta uma escrita, com duas escritas por troca. O tempo instrumentado mede o núcleo e seus contadores, após a cópia inicial. Não é o tempo total da chamada nem uma comparação com o sorted otimizado de Python.

BFS usa a fila própria e marca o estado ao enfileirá-lo. Explora camadas e encontra o menor número de transições no espaço de estados. Isso não garante menor energia quando as arestas têm custos diferentes. DFS usa pilha, marca na retirada e empilha vizinhos em ordem inversa para manter a ordem observável da versão recursiva. Busca um caminho válido, sem garantia de menor comprimento ou custo.

Na DFS recursiva, o primeiro caso-base rejeita uma chave repetida; o segundo reconhece o objetivo. A chamada recursiva explora sucessores e o retorno propaga sucesso ou fracasso. A AVL evita expansão repetida. O rastro registra chamadas, casos-base, sucesso e ramo sem saída.

Se V é o número de estados e E o de transições geradas, o índice AVL introduz custo O((V+E) log V), sob custo constante para construção e comparação de chaves. A AVL e os registros por estado usam O(V). Como a DFS iterativa marca na retirada, pode haver estados repetidos pendentes na pilha; o limite conservador de memória total é O(V+E). A versão recursiva também materializa rastro e sucessores. O custo real de ordenar coleções na chave cresce com o número de pistas/portas, além dessa simplificação.

6.1. Árvore de busca registrada

No grafo conhecido 0-1, 0-2, 1-3, 2-3, com objetivo 3, a BFS registrou visita [0, 1, 2, 3] e caminho 0->1->3. arvore_busca() retornou:

```
  0               Arestas pai/filho/acao:
 / \              (0, 1, '0->1')
1   2             (0, 2, '0->2')
|                 (1, 3, '1->3')
3
```

A árvore representa os pais escolhidos na exploração, não todas as arestas do grafo. O solver pode parar no primeiro objetivo; explorar todo o espaço não é necessário para encontrar uma solução.

7. Estratégia e resultados dos testes

Os testes são executados na .venv existente do projeto. A suíte original passou com 183 testes em 23,65 s. Depois da correção do grafo e da ampliação de testes de interface existentes, passou com 184 testes em 19,28 s. Com a inclusão explícita dos artefatos acadêmicos no ZIP e seu teste, passou com 185 testes em 25,08 s. Não houve exclusão ou desabilitação de testes para obter aprovação.

Grupo	Evidências verificadas
Registros e ordenação	Vazio, um elemento, duplicação de id, seed, chave composta, entrada preservada, ordenado/inverso/repetidos e equivalência dos três algoritmos.
Estruturas lineares	FIFO/LIFO, operações vazias, consultas, inserções e remoções no início/meio/fim, busca e elemento ausente.
Grafo e busca	Vértice isolado, aresta duplicada recusada, ciclos, caminhos alternativos, destino sem solução, ordem conhecida e caminho revalidado pelas regras.
Estados e AVL	Pistas diferentes na mesma posição, chave equivalente, busca existente/inexistente, quatro rotações e balanceamento após muitas inserções.
Regras e sessão	Portas, pistas, energia, vitória, reinício, selos, desfazer, ações inválidas e auditoria inconsistente após reinício.
Interface	Navegação, bloqueio, cinco fases, sessão independente, solver, evento do mapa, Resultados e downloads.

O defeito novo tinha uma reprodução mínima: inserir B->A como aresta dirigida e depois pedir A-B bidirecional. A segunda operação era recusada, mas deixava A->B inserida, com contagem incompatível. O teste comparou todas as adjacências antes/depois; falhou com o código anterior. A correção antecipa a validação da direção inversa antes de modificar a estrutura.

O servidor existente foi confirmado em http://127.0.0.1:8090. Início, Jogar, Fases 1 a 5, Resolver, Instruções, Resultados e Sair responderam HTTP 200. O teste de integração simula a partida e verifica que o selo decorre da sessão. O teste do mapa envia veridia_local ao listener registrado na raiz e verifica exatamente um movimento. Os botões continuam cobertos.

Resultados foi exercitado após auditoria consistente. Os botões de exportação produziram respostas reais de download na simulação: JSON consistente e HTML com o conteúdo esperado e estilo de impressão.

O controle do Chrome foi bloqueado por não determinar a URL atual com segurança. Não foi executado clique físico no SVG, revisão visual de Resultados nem impressão do HTML nesta rodada. HTTP e usuário simulado não comprovam esses aspectos. Esses itens continuam para validação manual.

8. Análise de complexidade e desempenho

8.1. Protocolo experimental

As medições foram realizadas em 06/11/2026 na mesma máquina, com a .venv do projeto: Python 3.11.9, NiceGUI 4.0.0, pytest 9.2.0 e pytest-asyncio 1.4.1.post1; Windows build 26200, processador AMD64 Family 25 Model 80 Stepping 0. Não foram instalados pacotes nesta rodada.

Foram executadas cinco repetições por combinação, sem aquecimento ou randomização da ordem. Ordenação: tamanhos 100, 500 e 1000, seed 2026, entradas aleatórias, ordenadas e inversas, três algoritmos: 135 execuções. Busca: energias 18, 15 e 14, três algoritmos, cinco repetições: 45 execuções. Os tempos são medidos por perf_counter; as tabelas usam a mediana.

Cada saída de ordenação foi conferida contra a ordem de chaves esperada pela rotina de experimento existente. Todos os caminhos encontrados foram reproduzidos por reproduzir_caminho e passaram na verificação de vitória. Os contadores estruturais foram iguais entre as cinco repetições de cada combinação. Os tempos brutos, mínimos e máximos estão no JSON de métricas.

8.2. Resultados da ordenação

As medições usam instâncias pequenas, execução instrumentada, ordem fixa e uma máquina com outros processos. Cinco repetições permitem observar variação local, mas não estimar desempenho de produção nem generalizar tempos para outros equipamentos. Nenhum teste de significância foi realizado e não há interpretação causal dessas diferenças.

n	Entrada	Algoritmo	Comparacoes	Escritas	Mediana ms
100	aleatorio	bubble_sort	4914	5312	4.296
100	aleatorio	merge_sort	531	1282	0.658
100	aleatorio	quick_sort	1122	380	1.105
100	ordenado	bubble_sort	99	0	0.093
100	ordenado	merge_sort	316	988	0.565
100	ordenado	quick_sort	1068	0	1.107
100	inverso	bubble_sort	4950	9900	5.012
100	inverso	merge_sort	356	1344	0.708
100	inverso	quick_sort	1069	104	0.847
500	aleatorio	bubble_sort	124540	133444	126.370
500	aleatorio	merge_sort	3847	8670	5.297
500	aleatorio	quick_sort	7005	2544	6.677
500	ordenado	bubble_sort	499	0	0.492
500	ordenado	merge_sort	2216	6704	3.416
500	ordenado	quick_sort	6484	0	5.442

n	Entrada	Algoritmo	Comparacoes	Escritas	Mediana ms
500	inverso	bubble_sort	124750	249500	120.880
500	inverso	merge_sort	2272	8976	3.083
500	inverso	quick_sort	6485	504	4.906
1000	aleatorio	bubble_sort	497222	515162	411.683
1000	aleatorio	merge_sort	8686	19283	11.812
1000	aleatorio	quick_sort	15146	5292	11.668
1000	ordenado	bubble_sort	999	0	0.645
1000	ordenado	merge_sort	4932	14908	7.311
1000	ordenado	quick_sort	13975	4	10.247
1000	inverso	bubble_sort	499500	998996	450.159
1000	inverso	merge_sort	5045	19951	5.894
1000	inverso	quick_sort	13976	1008	9.369

Em entrada aleatória com n=1000, o Bubble executa aproximadamente $n^2/2$ comparações. Merge e Quick apresentam escala compatível com n log n neste experimento. Em entrada ordenada, a parada antecipada do Bubble reduz o contador a n-1 e nenhuma movimentação. Isso não elimina seu pior caso quadrático em entradas inversas.

As contagens não devem ser interpretadas como custos idênticos de CPU: uma comparação chama a função de chave, enquanto uma movimentação escreve uma referência. O Merge faz menos comparações nessas entradas e usa um vetor auxiliar. O Quick troca menos referências. Não há evidência aqui de superioridade universal de um algoritmo.

8.3. Resultados das buscas e AVL

Energia	Busca	Mov.	Custo	Expandidos	Fronteira	Mediana ms
18	BFS	8	15	308	135	22.593
18	DFS	10	17	188	15	7.171
18	DFS_RECURSIVA	10	17	188	12	9.165
15	BFS	8	15	259	82	17.173
15	DFS	8	15	139	12	5.602
15	DFS_RECURSIVA	8	15	139	10	7.533
14	BFS	-	-	249	66	16.640
14	DFS	-	-	249	14	14.307
14	DFS_RECURSIVA	-	-	249	15	16.085

Com energia 18, a BFS encontra oito movimentos e custo 15, enquanto as DFS encontram dez movimentos e custo 17. As DFS expandem menos estados nessa ordem de vizinhança. Com energia 15, os três algoritmos encontram oito movimentos de custo 15; com 14, nenhum alcança um objetivo e todos expandem 249 estados. A fronteira recursiva representa profundidade de chamada e não uma fila equivalente à da BFS.

Energia	Busca	AVL n	Altura	Rot. esq.	Rot. dir.	Dupla E-D / D-E
18	BFS	425	11	132	170	48 / 41
18	DFS	188	9	75	53	11 / 25
18	DFS_RECURSIVA	188	9	75	53	11 / 25
15	BFS	302	10	94	115	32 / 26
15	DFS	139	8	61	43	9 / 25
15	DFS_RECURSIVA	139	8	61	43	9 / 25
14	BFS	249	9	78	100	32 / 22
14	DFS	249	9	128	99	30 / 43
14	DFS_RECURSIVA	249	9	128	99	30 / 43

A BFS pode registrar estados na fronteira antes de expandi-los; por isso o total da AVL pode superar os estados expandidos quando a busca para no objetivo. Os contadores de rotação descrevem operações de balanceamento, não movimentos do jogador. As rotações duplas são também contabilizadas nas simples componentes.

O espaço máximo de combinações é limitado por número de locais vezes 2 elevado ao número de pistas, vezes 2 elevado ao número de portas, vezes (energia inicial+1). Muitas combinações são inalcançáveis, mas o produto ilustra por que condições adicionais ampliam a busca. A energia na chave permite retornos ao mesmo local com menos recursos. As curvas teóricas do algoritmo e o crescimento do espaço são questões distintas.

9. Uso de IAG e curadoria humana

O desenvolvimento utilizou Claude Code nas rodadas 1 a 3 e Codex nesta rodada. O histórico em REGISTRO_IA.md documenta objetivos, escolhas, correções e testes. As interações abaixo foram recuperadas desse registro e desta sessão; não são transcrições inventadas de prompts.

Ferramenta/data	Objetivo	Ideia ou código aproveitado	Alterações documentadas e autoria	Forma de validação
Claude Code, 05/11/2026	Modelar dados	Registros imutáveis, chave canônica e seed.	IA gerou os modelos; alterações individuais do aluno não detalhadas.	Reprodução e equivalência de estados.
Claude Code, 05/11/2026	Ordenação	Bubble/Merge/Quick com métricas.	IA adotou Hoare e mediana de três e corrigiu contagem do Merge.	Casos-limite, estabilidade e fórmulas de contagem.
Claude Code, 05/11/2026	Grafo e solver	Regras compartilhadas e AVL.	IA acrescentou porta Arquivo-Ponte para impedir desvio.	Caminhos reexecutados e cenário sem solução.
Claude Code, 05/11/2026	Integrar fases	Sessão e auditoria do estado atual.	IA corrigiu a origem identificada para a trilha.	Testes de sessão e auditoria.
Claude Code, 05/11/2026	Interface	UI por rotas e visualizações.	IA corrigiu refresh assíncrono e registro de eventos.	Usuário simulado e capturas registradas na Rodada 3.
Codex, 05-06/11/2026	Entrega oficial	PDF confrontado com código.	IA consolidou rastreabilidade e relatório acadêmico.	Leitura das 20 páginas e evidências nos módulos/testes.
Codex, 06/11/2026	Defeito e entrega	Validação atômica do grafo e pacote acadêmico.	IA antecipou validação, ampliou testes e inclusões do ZIP.	Regressão falhou antes da correção; suíte passou com 185 testes.

A seção 14.4 do enunciado pede também as alterações feitas pelo estudante. O histórico não detalha esse campo para cada interação. Não é correto atribuir à pessoa as mudanças executadas pela IA. Há registro de revisão externa do código e reexecução de 89 testes pelo aluno entre rodadas, mas isso não demonstra a curadoria individual de todas as decisões.

Antes da entrega, o estudante deve complementar pelo menos cinco linhas com suas alterações ou sua decisão fundamentada de manter o material, registrar como o validou pessoalmente e confirmar a compreensão dos algoritmos. As confirmações dadas na sessão sobre localização do PDF e repositório de publicação não equivalem a essa validação acadêmica.

10. Dificuldades, limitações, melhorias e conclusão

O histórico registra dificuldades concretas: uma porta inicialmente contornável no mapa, mensagens consumidas por renderizações assíncronas, handlers acumulados e inconsistência entre trilha manual e trilha de referência. Essas situações motivaram correções acompanhadas de testes. Nesta rodada, o conflito de aresta dirigida/bidirecional demonstrou a importância de validar a operação integral antes de alterar a estrutura.

A BFS minimiza movimentos; a implementação não inclui busca de menor custo. A DFS recursiva depende do limite de recursão de Python. A energia na chave aumenta o espaço alcançável. As partidas são mantidas somente em memória. Esses limites foram preservados porque o objetivo é atender ao EP com soluções verificáveis, sem refatoração ampla ou extensões bônus.

Do ponto de vista técnico, a implementação integra as cinco fases, estruturas próprias, ordenações, recursão, controle AVL e solver com as mesmas regras do modo manual. Os testes e os experimentos sustentam a corretude nos cenários exercitados, sem provar ausência de todo defeito. O relatório acadêmico registra evidências e limitações para que o aluno possa revisá-las, explicá-las e assumir as decisões de entrega.

10.1. Revisão pessoal ainda necessária

- Confirmar identificação acadêmica, autoria e entendimento de cada módulo.

- Acrescentar considerações pessoais sobre dificuldades, decisões mantidas e aprendizado; essas experiências não podem ser redigidas como se já tivessem sido relatadas pelo estudante.

- Completar a curadoria humana da tabela de IAG exigida na seção 14.4.

- Conferir clique físico no mapa, visual de Resultados e abertura/impressão dos relatórios no navegador; executar uma partida manual e o solver em sala. Um teste com pessoa que não programou o jogo é recomendado.

- Revisar este Markdown e o PDF antes da entrega. A versão é tecnicamente fundamentada, mas não está homologada pelo estudante ou pela professora.

10.2. Publicação e critérios de avaliação

O projeto pertence ao repositório data-science-projects. A branch atual contém histórico de outros projetos; a estratégia proposta é uma branch própria a partir de origin/main, integrada em worktree isolada após aprovação. Não foram feitos commit ou push nesta rodada. CI é preparação de manutenção, não substitui teste local ou demonstração em sala.

A seção 17 do PDF atribui pesos 20 para funcionalidade, 20 para estruturas, 20 para algoritmos, 20 para grafo/estados/AVL, 15 para testes/análise, 15 para criatividade/experiência e 10 para relatório/curadoria. Esses valores somam 120 no documento; foram preservados, sem normalização ou suposição sobre a composição da nota.

Referências e rastreabilidade

IPT. EP Urna Lógica: O Enigma da Eleição de Verídia. Enunciado oficial da disciplina, 20 páginas. Consultado em 06/11/2026; sem atribuir uma data editorial não identificada no arquivo.

VERÍDIA. Código Python em veridia/, testes em tests/ e documentação em docs/. Estado local da Rodada 4, anterior ao commit inicial. A matriz REQUISITOS_EP.md aponta páginas/seções, módulos e evidências. O registro METRICAS_RODADA4.json permite inspecionar as medições apresentadas.